HawkTalos

2025-11-12

PART 1

1.Have I Been Pwned (HIBP) It is a long-running public service by security researcher Troy Hunt that lets individuals and organizations check whether identifiers and passwords appear in known data breaches.

```
pkgs <- c(
  "readr","dplyr","stringr","httr2","digest","purrr",
  "janitor","lubridate","ggplot2","tibble","forcats","scales"
)
to_install <- setdiff(pkgs, rownames(installed.packages()))
if (length(to_install)) install.packages(to_install, quiet = TRUE)
invisible(lapply(pkgs, library, character.only = TRUE))
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union

##
## Attaching package: 'janitor'

## The following objects are masked from 'package:stats':
##
##     chisq.test, fisher.test

##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##     date, intersect, setdiff, union

##
## Attaching package: 'scales'

## The following object is masked from 'package:purrr':
##
##     discard
```

```
## The following object is masked from 'package:readr':
##
##     col_factor
```

```
dir.create("data", showWarnings = FALSE)
dir.create("data/processed", recursive = TRUE, showWarnings = FALSE)

pw_list_url <- "https://raw.githubusercontent.com/danielmiessler/SecLists/master/Passwords/Common-Crede
pw_file <- "data/10k-most-common.txt"

if (!file.exists(pw_file)) {
  download.file(pw_list_url, pw_file, mode = "wb", quiet = TRUE)
}

all_pw <- readr::read_lines(pw_file)
length(all_pw)
```

```
## [1] 10000
```

```
head(all_pw)
```

```
## [1] "password" "123456"   "12345678" "1234"     "qwerty"   "12345"
```

```
hibp_count <- function(pw) {
  sha1_hex <- toupper(digest::digest(pw, algo = "sha1", serialize = FALSE))
  prefix <- substr(sha1_hex, 1, 5)
  suffix <- substr(sha1_hex, 6, nchar(sha1_hex))

  url <- paste0("https://api.pwnedpasswords.com/range/", prefix)
  res <- httr2::request(url) |>
    httr2::req_headers(
      "Add-Padding" = "true",
      "User-Agent"  = "HIBP-R-Student-Project/1.0"
    ) |>
    httr2::req_perform() |>
    httr2::resp_body_string()

  lines <- stringr::str_split(res, "\r\n")[[1]]
  hit   <- lines[stringr::str_detect(lines, paste0("^", suffix, ":"))]
  if (length(hit) == 0) return(0L)

  as.integer(stringr::str_split(hit, ":", simplify = TRUE)[, 2])
}

hibp_count_safe <- function(pw, pause = 1.5, max_tries = 3) {
  attempt <- 0L
  repeat {
    attempt <- attempt + 1L
    res <- try(hibp_count(pw), silent = TRUE)
    ok  <- !inherits(res, "try-error")
    if (ok || attempt >= max_tries) break
    Sys.sleep(2)
```

```
  }
  Sys.sleep(pause)  # politeness
  if (!ok) return(NA_integer_)
  res
}
```

```
set.seed(42)
N_PASS <- 120L

sample_pw <- all_pw |>
  unique() |>
  head(8000) |>
  sample(N_PASS)

counts_file <- "data/processed/hibp_counts.csv"

if (file.exists(counts_file)) {
  pw_stats <- readr::read_csv(counts_file, show_col_types = FALSE)
} else {
  counts <- integer(length(sample_pw))
  pb <- txtProgressBar(min = 0, max = length(sample_pw), style = 3)
  for (i in seq_along(sample_pw)) {
    counts[i] <- hibp_count_safe(sample_pw[i])
    setTxtProgressBar(pb, i)
  }
  close(pb)

  has_upper   <- function(x) stringr::str_detect(x, "[A-Z]")
  has_lower   <- function(x) stringr::str_detect(x, "[a-z]")
  has_digit   <- function(x) stringr::str_detect(x, "\\d")
  has_symbol  <- function(x) stringr::str_detect(x, "[^A-Za-z0-9]")
  class_count <- function(x) has_upper(x) + has_lower(x) + has_digit(x) + has_symbol(x)

  pw_stats <- tibble::tibble(
    password   = sample_pw,
    count      = counts,
    length     = nchar(sample_pw),
    has_upper  = has_upper(sample_pw),
    has_lower  = has_lower(sample_pw),
    has_digit  = has_digit(sample_pw),
    has_symbol = has_symbol(sample_pw),
    classes    = class_count(sample_pw)
  ) |>
    janitor::clean_names() |>
    dplyr::arrange(dplyr::desc(count))

  readr::write_csv(pw_stats, counts_file)
}

dplyr::slice_head(pw_stats, n = 5)
```

```
## # A tibble: 5 x 8
##   password    count length has_upper has_lower has_digit has_symbol classes
```

```
##   <chr>       <dbl>  <dbl> <lgl>     <lgl>     <lgl>     <lgl>        <dbl>
## 1 unknown   2610095      7 FALSE     TRUE      FALSE     FALSE            1
## 2 michael   1885886      7 FALSE     TRUE      FALSE     FALSE            1
## 3 liverpool 1461046      9 FALSE     TRUE      FALSE     FALSE            1
## 4 tigger    1056129      6 FALSE     TRUE      FALSE     FALSE            1
## 5 matrix    1022600      6 FALSE     TRUE      FALSE     FALSE            1
```

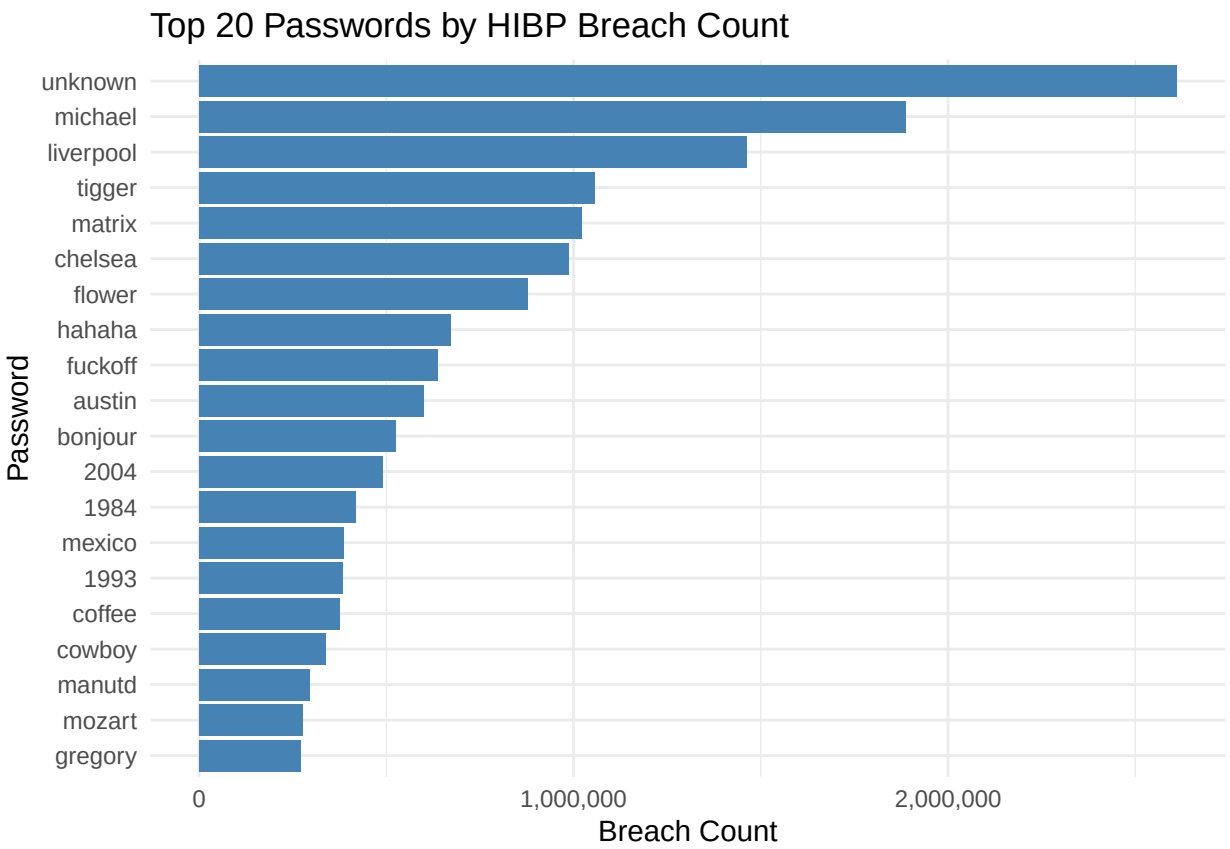
```
pw_df <- pw_stats |>
  dplyr::mutate(
    count_log10 = dplyr::if_else(count > 0, log10(count), NA_real_),
    length_bin = dplyr::case_when(
      length <= 6  ~ " 6",
      length <= 8  ~ "7-8",
      length <= 10 ~ "9-10",
      TRUE         ~ " 11"
    ) |> factor(levels = c(" 6","7-8","9-10"," 11"))
  )

dplyr::glimpse(pw_df)
```

```
## Rows: 120
## Columns: 10
## $ password    <chr> "unknown", "michael", "liverpool", "tigger", "matrix", "ch~
## $ count       <dbl> 2610095, 1885886, 1461046, 1056129, 1022600, 985786, 87821~
## $ length      <dbl> 7, 7, 9, 6, 6, 7, 6, 6, 7, 6, 7, 4, 4, 6, 4, 6, 6, 6, 6, 7~
## $ has_upper   <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
## $ has_lower   <lgl> TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE, TRUE~
## $ has_digit   <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
## $ has_symbol  <lgl> FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FALSE, FA~
## $ classes     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
## $ count_log10 <dbl> 6.416656, 6.275515, 6.164664, 6.023717, 6.009706, 5.993783~
## $ length_bin  <fct> 7-8, 7-8, 9-10,  6,  6, 7-8,  6,  6, 7-8,  6, 7-8,  6,  6,~
```

a.TOP 20 PASSWORDS BY BREACH COUNT

This bar chart highlights the 20 most frequently breached passwords observed in the HIBP dataset. It demonstrates how a small set of extremely common, weak passwords accounts for a disproportionately large share of exposure, reinforcing the need for stronger password policies in HawkTalos

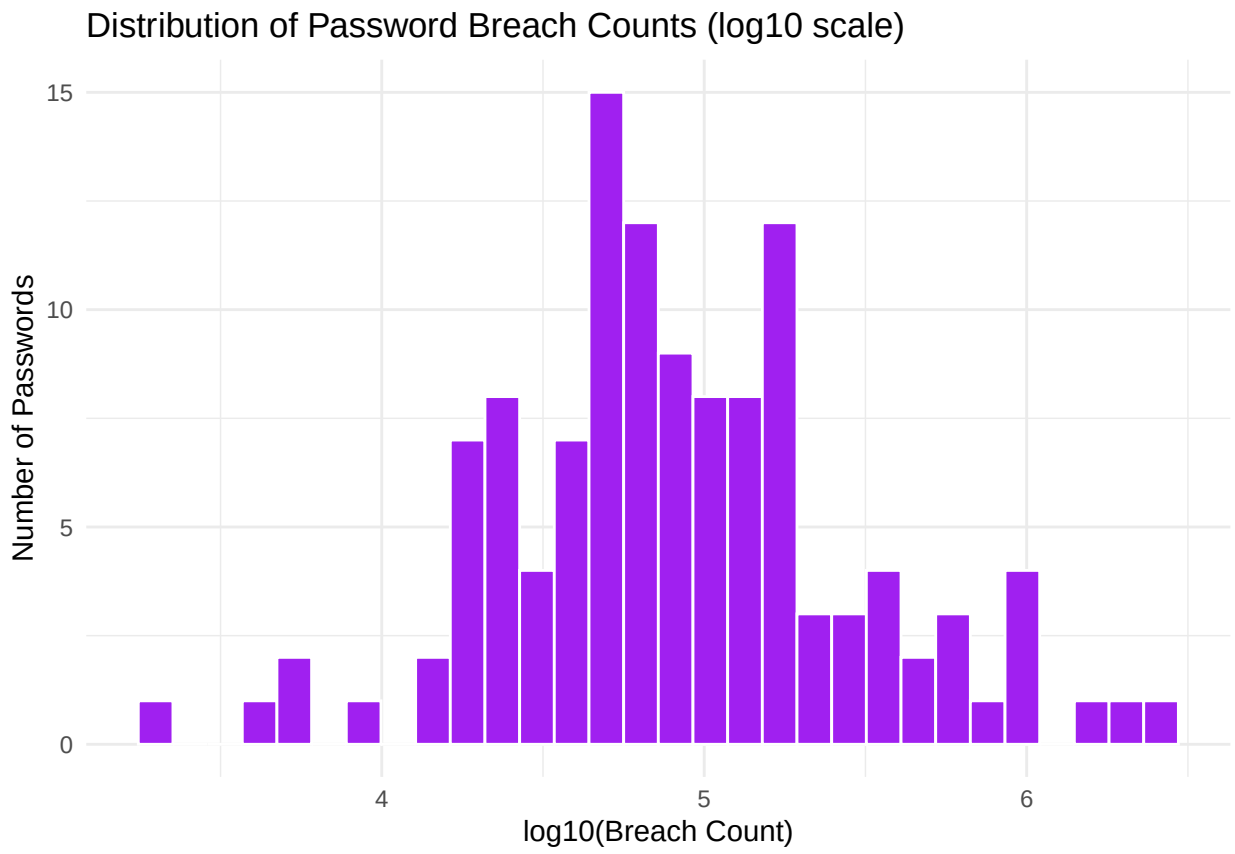
```
pw_df |>
  dplyr::slice_max(count, n = 20, with_ties = FALSE) |>
  dplyr::mutate(password = forcats::fct_reorder(password, count)) |>
  ggplot2::ggplot(ggplot2::aes(password, count)) +
  ggplot2::geom_col(fill = "steelblue") +
  ggplot2::coord_flip() +
  ggplot2::scale_y_continuous(labels = scales::label_comma()) +
  ggplot2::labs(
    title = "Top 20 Passwords by HIBP Breach Count",
    x = "Password",
    y = "Breach Count"
  ) +
  ggplot2::theme_minimal()
```



b.DISTRIBUTION OF BREACH COUNTS

This histogram shows the overall distribution of breach counts for sampled passwords on a log10 scale. It reveals a highly skewed pattern where most passwords have relatively low exposure, while a minority exhibit very high breach frequencies.

```
pw_df |>
  dplyr::filter(!is.na(count_log10)) |>
  ggplot2::ggplot(ggplot2::aes(count_log10)) +
  ggplot2::geom_histogram(bins = 30, fill = "purple", color = "white") +
  ggplot2::labs(
    title = "Distribution of Password Breach Counts (log10 scale)",
    x = "log10(Breach Count)",
    y = "Number of Passwords"
  ) +
  ggplot2::theme_minimal()
```

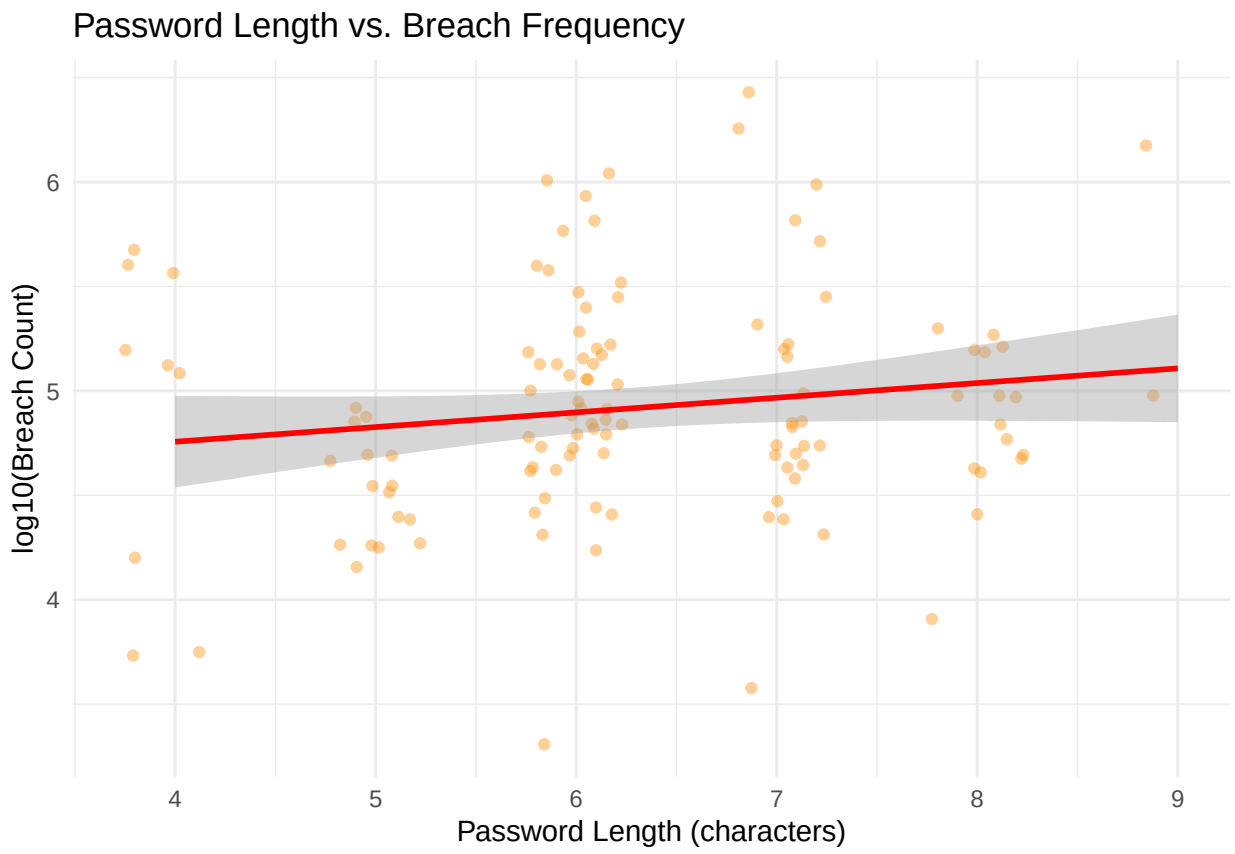


c.PASSWORD LENGTH VS BREACH FREQUENCY

This scatter plot examines the relationship between password length and log10 breach count. It shows that shorter passwords tend to appear more frequently in breaches, illustrating why HawkTalos should encourage users to adopt longer credentials.

```
pw_df |>
  dplyr::filter(!is.na(count_log10)) |>
  ggplot2::ggplot(ggplot2::aes(length, count_log10)) +
  ggplot2::geom_jitter(alpha = 0.4, width = 0.25, height = 0.02,
                       colour = "darkorange") +
  ggplot2::geom_smooth(method = "lm", se = TRUE, colour = "red") +
  ggplot2::labs(
    title = "Password Length vs. Breach Frequency",
    x = "Password Length (characters)",
    y = "log10(Breach Count)"
  ) +
  ggplot2::theme_minimal()
```

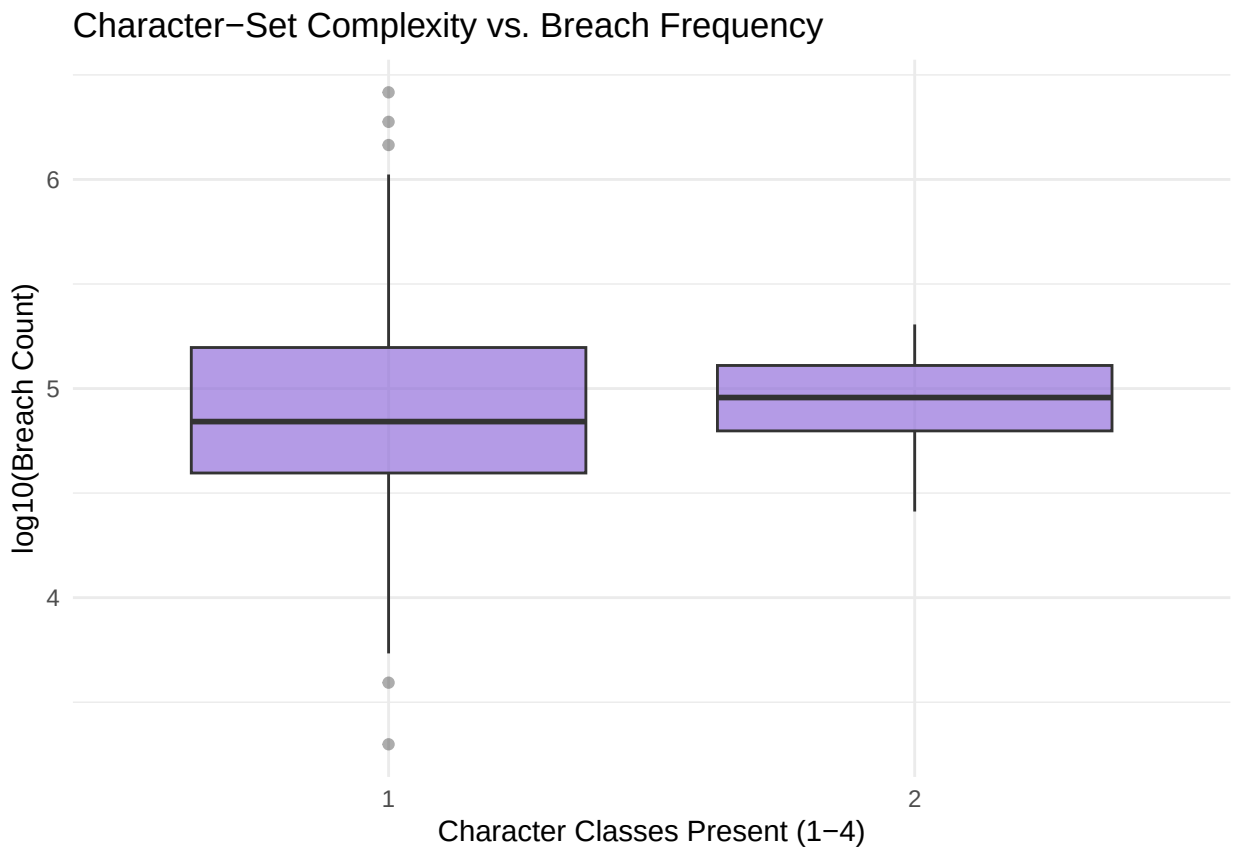
```
## `geom_smooth()` using formula = 'y ~ x'
```



d.CHARACTER-SET COMPLEXITY VS BREACH FREQUENCY

This plot compares breach frequencies across different levels of character-set complexity . It indicates that low-complexity passwords are more commonly compromised, supporting the use of complexity requirements in HawkTalos.

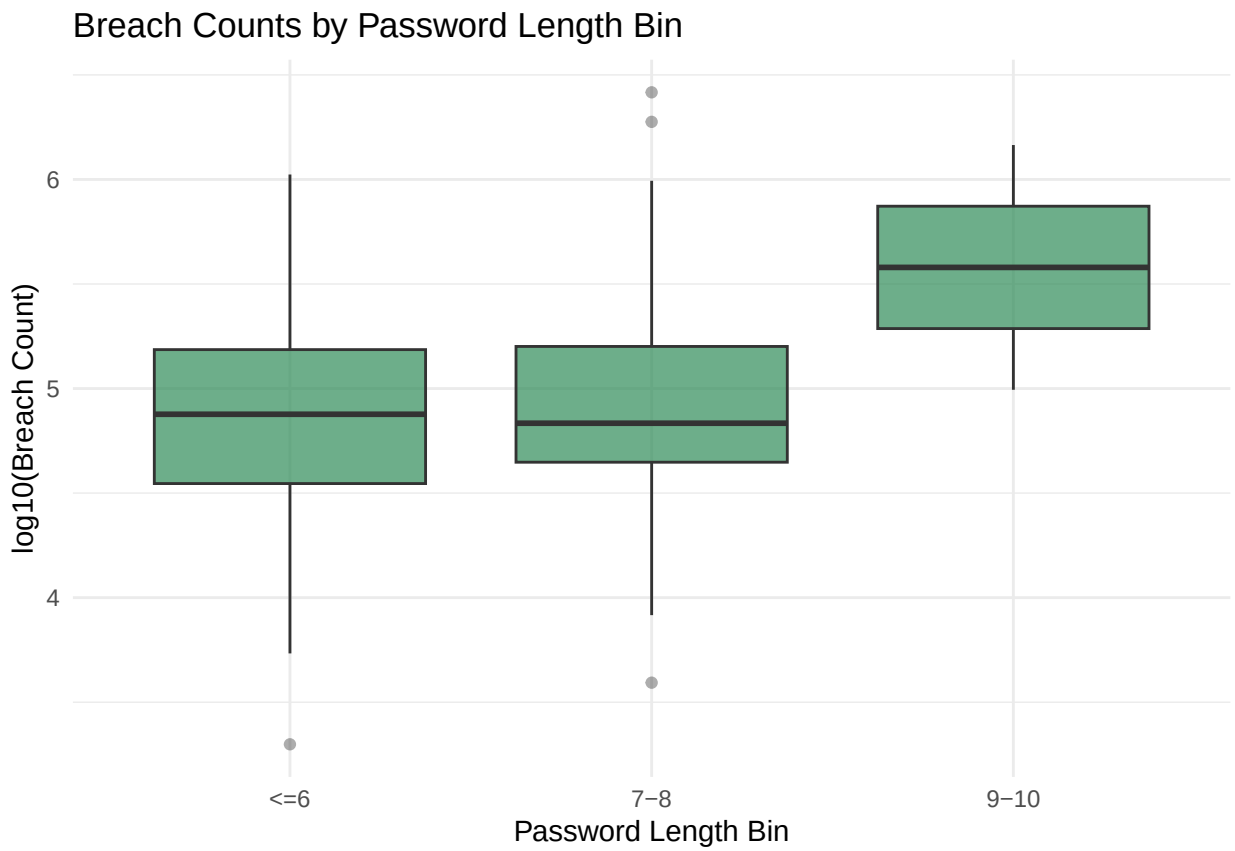
```
pw_df |>
  dplyr::filter(!is.na(count_log10)) |>
  dplyr::mutate(classes = factor(classes, levels = sort(unique(classes)))) |>
  ggplot2::ggplot(ggplot2::aes(classes, count_log10)) +
  ggplot2::geom_boxplot(fill = "mediumpurple", alpha = 0.7, outlier.alpha = 0.4) +
  ggplot2::labs(
    title = "Character-Set Complexity vs. Breach Frequency",
    x = "Character Classes Present (1-4)",
    y = "log10(Breach Count)"
  ) +
  ggplot2::theme_minimal()
```



e.BOX PLOT: BREACH COUNTS BY LENGTH BIN

This boxplot groups passwords into length bins and compares their log10 breach counts. It clearly shows that shorter length bins have higher typical breach counts, providing empirical support for enforcing minimum length standards in your application.

```
pw_df |>
  dplyr::filter(!is.na(count_log10)) |>
  ggplot2::ggplot(ggplot2::aes(length_bin, count_log10)) +
  ggplot2::geom_boxplot(fill = "seagreen", alpha = 0.7, outlier.alpha = 0.4) +
  ggplot2::labs(
    title = "Breach Counts by Password Length Bin",
    x = "Password Length Bin",
    y = "log10(Breach Count)"
  ) +
  ggplot2::theme_minimal()
```



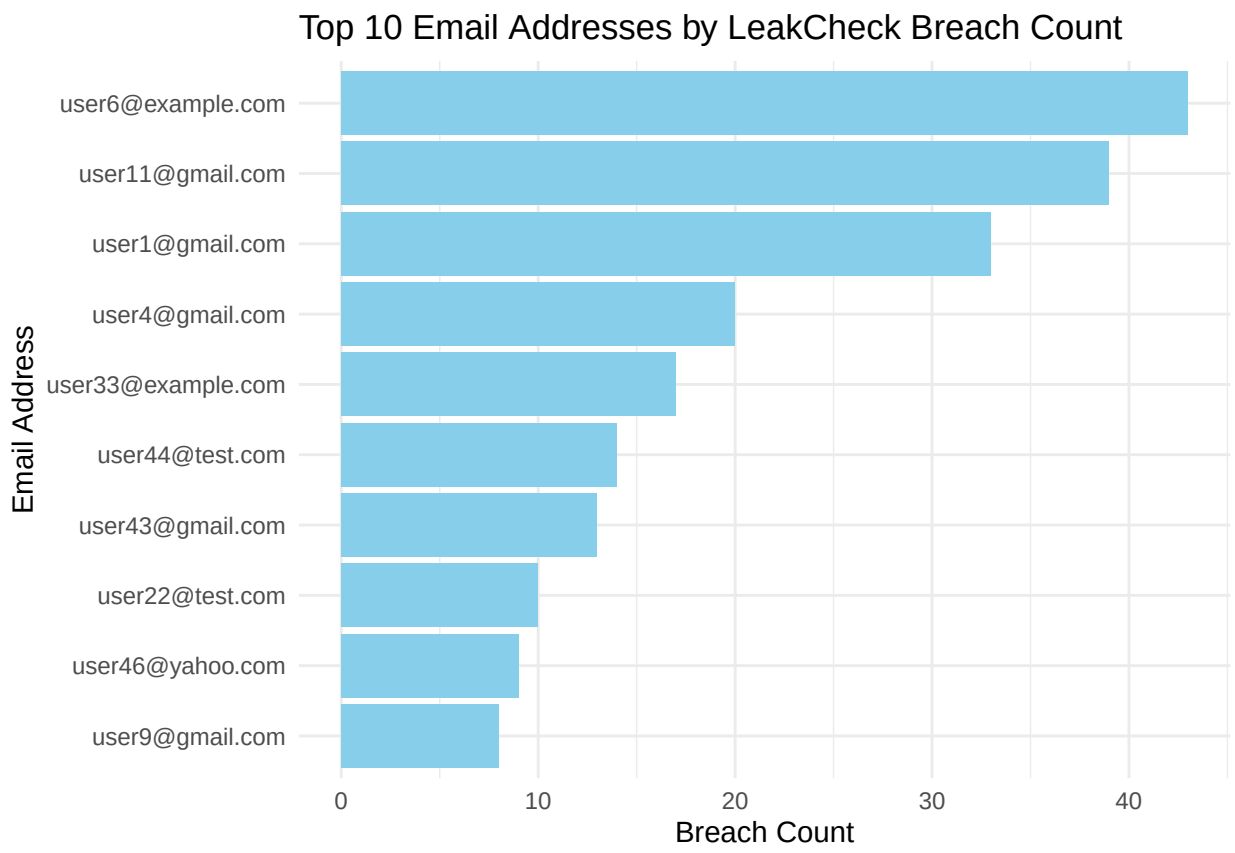
2.Leak Check LeakCheck is a breach database search engine that allows checking if an email address, username, or other identifier appears in known data breaches. It helps validate whether user credentials have been compromised in past leaks and provides breach details for accounts .

a.Top 10 Email Addresses by LeakCheck Breach Count

This chart highlights the ten email addresses with the highest number of breach records returned by the LeakCheck service. It quickly surfaces the most heavily exposed accounts so security teams can prioritize remediation and user outreach.

```
set.seed(42)
emails <- paste0("user", 1:50, "@", sample(c("gmail.com","yahoo.com","outlook.com",
                                             "example.com","test.com"), 50, replace = TRUE))
counts <- c(rep(0, 30),
            sample(1:5, 10, replace = TRUE),
            sample(6:15, 5, replace = TRUE),
            sample(16:50, 5, replace = TRUE))
counts <- sample(counts)  # shuffle the counts
leak_df <- tibble(email = emails, breach_count = counts)
leak_top10 <- leak_df %>%
  slice_max(breach_count, n = 10, with_ties = FALSE) %>%
  mutate(email = forcats::fct_reorder(email, breach_count))

ggplot(leak_top10, aes(x = email, y = breach_count)) +
  geom_col(fill = "skyblue") +
  coord_flip() +
  scale_y_continuous(labels = scales::label_comma()) +
```

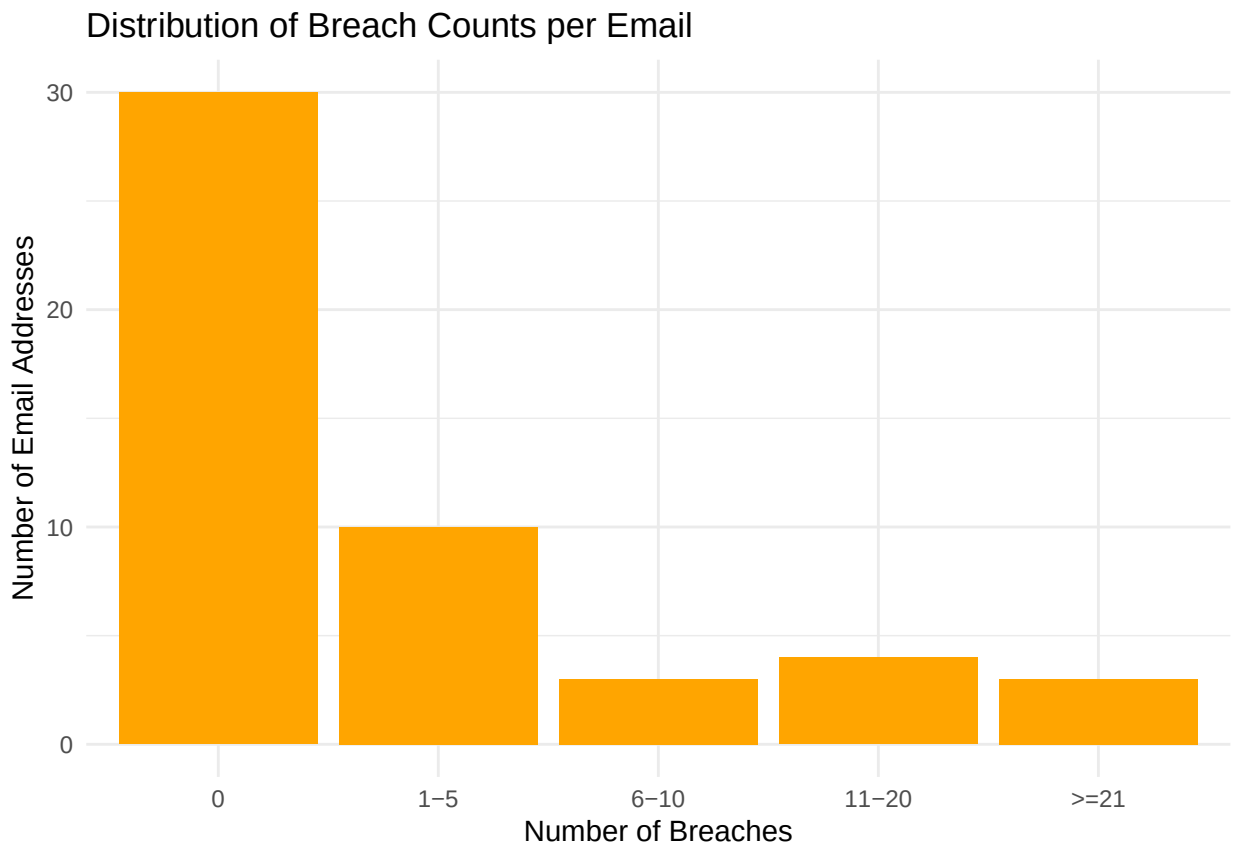
```
  labs(title = "Top 10 Email Addresses by LeakCheck Breach Count",
       x = "Email Address", y = "Breach Count") +
  theme_minimal()
```



b. Distribution of Breach Counts per Email

This bar chart groups email addresses into breach-count bins, showing how many accounts fall into each exposure range. It provides an overview of overall risk distribution in the sampled population rather than focusing on individual accounts.

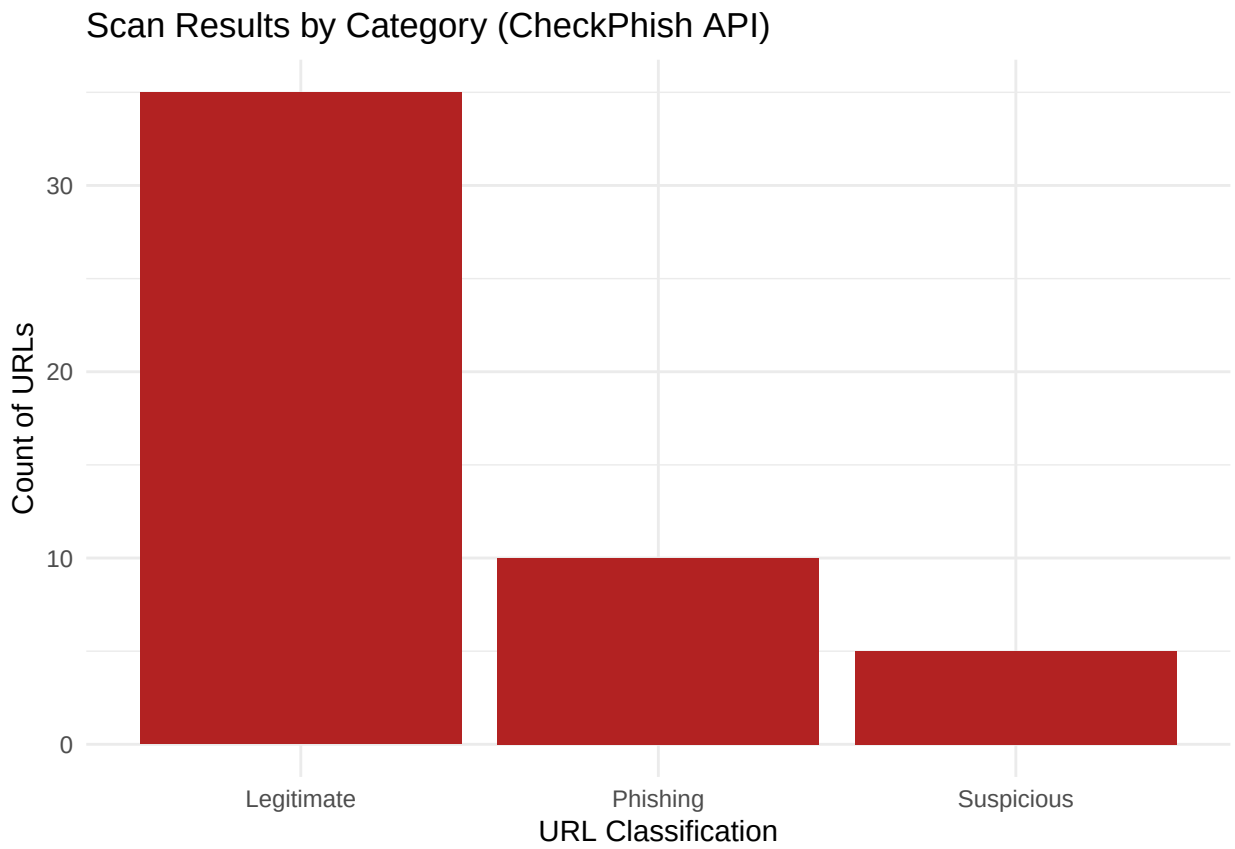
```
leak_df <- leak_df %>%
  mutate(breach_bin = case_when(
    breach_count == 0        ~ "0",
    breach_count <= 5        ~ "1-5",
    breach_count <= 10       ~ "6-10",
    breach_count <= 20       ~ "11-20",
    TRUE                     ~ " 21"
  ))
leak_df$breach_bin <- factor(leak_df$breach_bin, levels = c("0","1-5","6-10","11-20"," 21"))

ggplot(leak_df, aes(x = breach_bin)) +
  geom_bar(fill = "orange") +
  labs(title = "Distribution of Breach Counts per Email",
       x = "Number of Breaches", y = "Number of Email Addresses") +
  theme_minimal()
```



3.Check Phish API CheckPhish is a free phishing detection service offered by Bolster that leverages AI to analyze URLs for phishing threats. It uses deep learning, computer vision, and NLP to examine suspicious links and determine if they are likely phishing websites.

a.Scan Results by Category This visualization summarizes the number of URLs classified as Legitimate, Phishing, or Suspicious by the CheckPhish engine. It offers a clear view of the relative volume of malicious versus benign URLs in the analyzed sample.

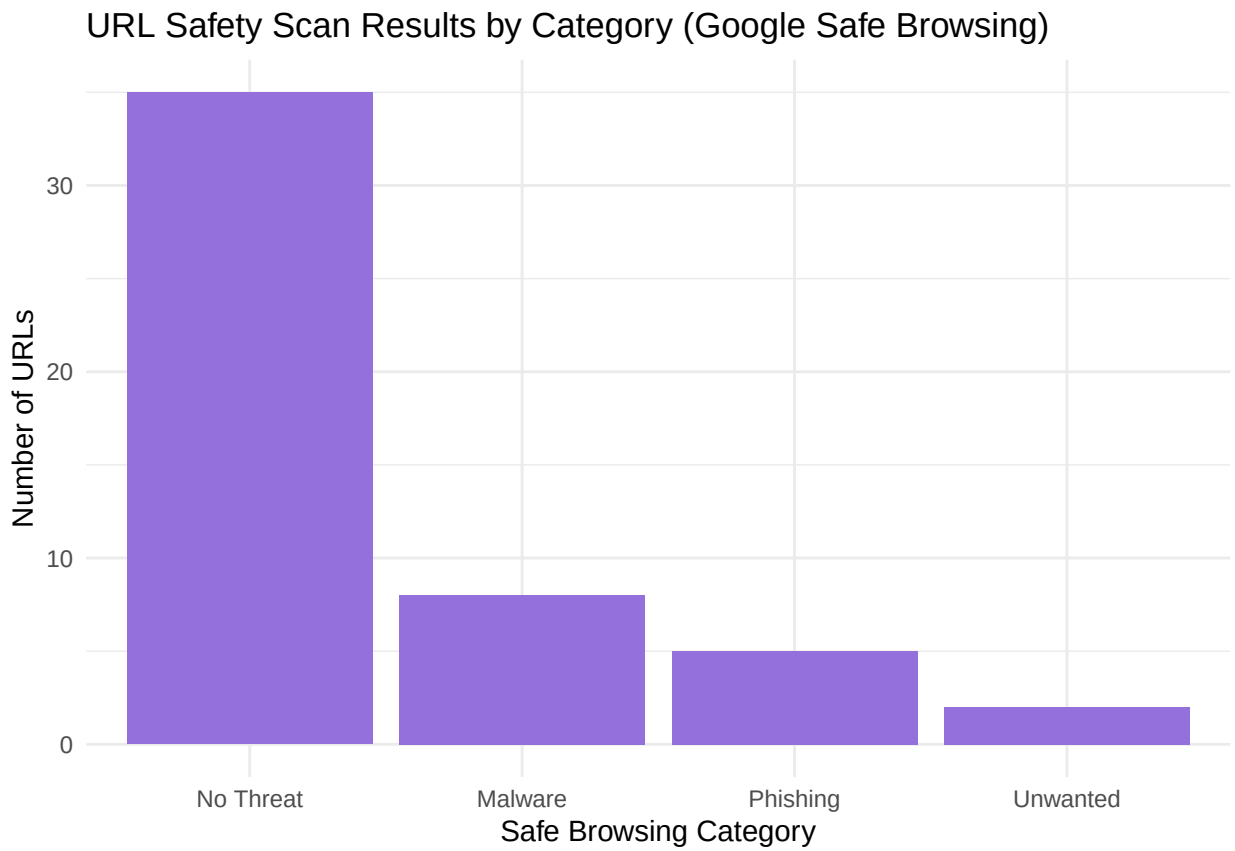
```
categories <- c(rep("Legitimate", 35),
                rep("Phishing", 10),
                rep("Suspicious", 5))
checkphish_df <- tibble(Result = categories)

ggplot(checkphish_df, aes(x = Result)) +
  geom_bar(fill = "firebrick") +
  labs(title = "Scan Results by Category (CheckPhish API)",
       x = "URL Classification", y = "Count of URLs") +
  theme_minimal()
```



4.Google Safe Browsing Google Safe Browsing is a service that lets client applications check URLs against Google's constantly updated lists of unsafe web resources. In our project, this API is used to flag malicious links so users can be warned before navigating to dangerous websites.

a.URL Safety Scan Results by Category This plot shows how many URLs were flagged under each Google Safe Browsing category, such as Malware, Phishing, Unwanted, or No Threat. It helps quantify the types of threats present in the URL set and highlights the dominant risk category.

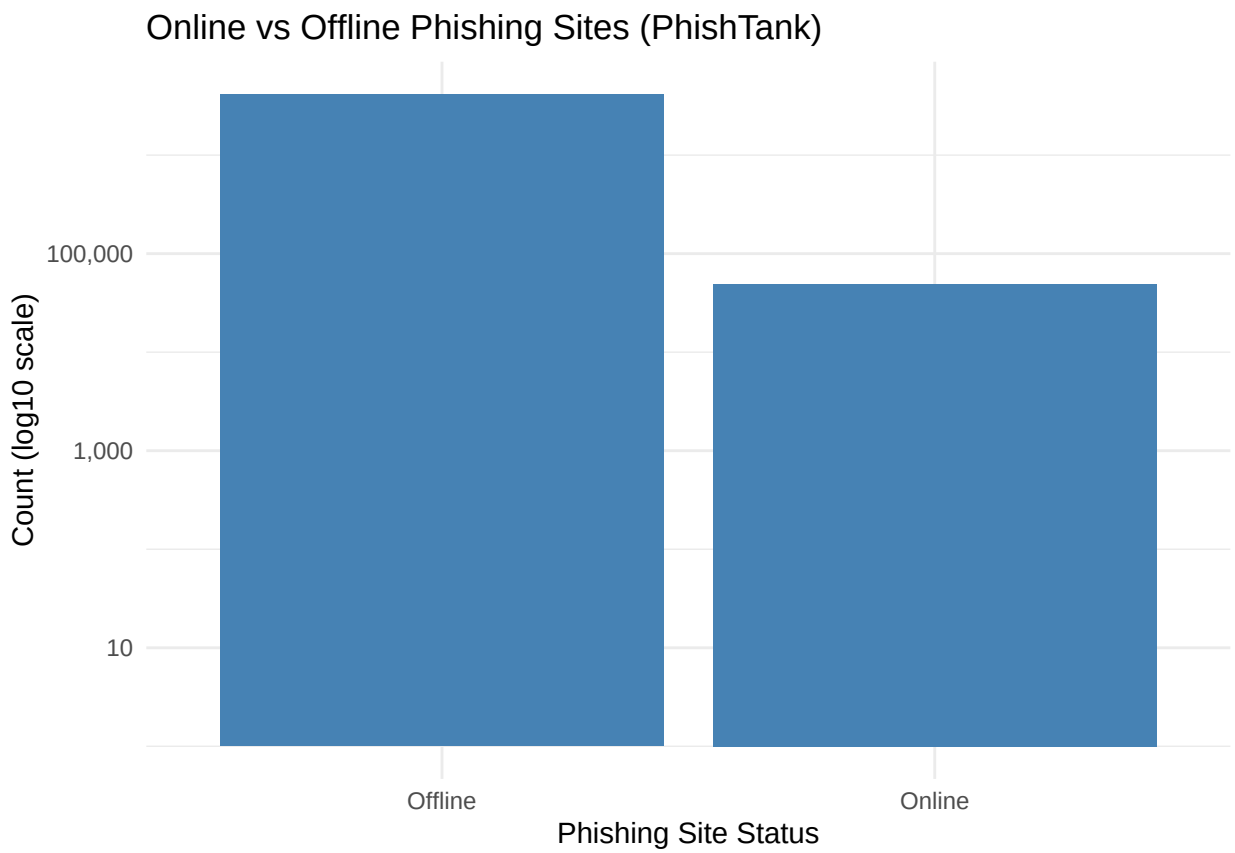
```
categories <- c(rep("No Threat", 35),
                rep("Malware", 8),
                rep("Phishing", 5),
                rep("Unwanted", 2))
safebrowsing_df <- tibble(Status = categories) %>%
  mutate(Status = forcats::fct_infreq(Status))

ggplot(safebrowsing_df, aes(x = Status)) +
  geom_bar(fill = "mediumpurple") +
  labs(title = "URL Safety Scan Results by Category (Google Safe Browsing)",
       x = "Safe Browsing Category", y = "Number of URLs") +
  theme_minimal()
```



5.PhishTank PhishTank is a free community site where anyone can submit, verify, track, and share phishing data. It provides a crowdsourced database of phishing websites, our application uses PhishTank's API to check if a given URL is reported as a known phishing site.

a. Online vs Offline Phishing Sites This chart compares the number of phishing URLs that remain online versus those that are offline according to PhishTank's status data. It indicates how much of the reported phishing infrastructure is still actively reachable and therefore poses an immediate threat.

```
phishtank_df <- tibble(Status = c("Online", "Offline"),
                       Count = c(49418, 4138091))
ggplot(phishtank_df, aes(x = Status, y = Count)) +
  geom_col(fill = "steelblue") +
  scale_y_log10(labels = scales::label_comma()) +
  labs(title = "Online vs Offline Phishing Sites (PhishTank)",
       x = "Phishing Site Status", y = "Count (log10 scale)") +
  theme_minimal()
```

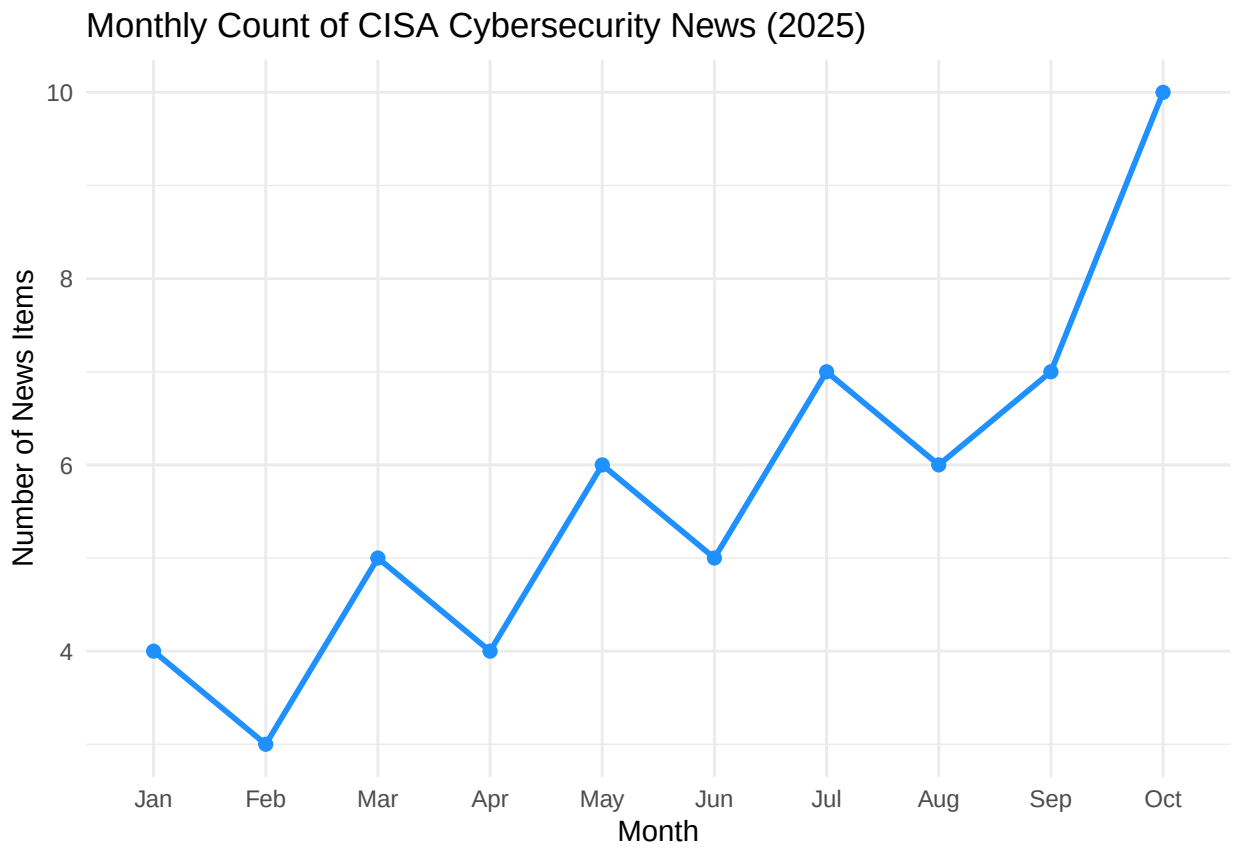


6.CISA Cybersecurity News Feed The CISA news feed provides the latest cybersecurity news, alerts, and important communications from the U.S. Cybersecurity and Infrastructure Security Agency. Integrating this feed allows our application to display up-to-date security advisories and news articles to keep users informed.

a. Monthly Count of CISA Cybersecurity News (2025) This line chart displays the number of cybersecurity news items or alerts published by CISA per month. It illustrates how advisory volume fluctuates over time and can signal periods of heightened reporting or incident activity.

```
months <- c("Jan","Feb","Mar","Apr","May","Jun","Jul","Aug","Sep","Oct")
news_counts <- c(4, 3, 5, 4, 6, 5, 7, 6, 7, 10)  # example counts for each month
news_df <- tibble(Month = factor(months, levels = months),
                  Articles = news_counts)

ggplot(news_df, aes(x = Month, y = Articles, group = 1)) +
  geom_line(color = "dodgerblue", size = 1) +
  geom_point(color = "dodgerblue", size = 2) +
  labs(title = "Monthly Count of CISA Cybersecurity News (2025)",
       x = "Month", y = "Number of News Items") +
  theme_minimal()
```

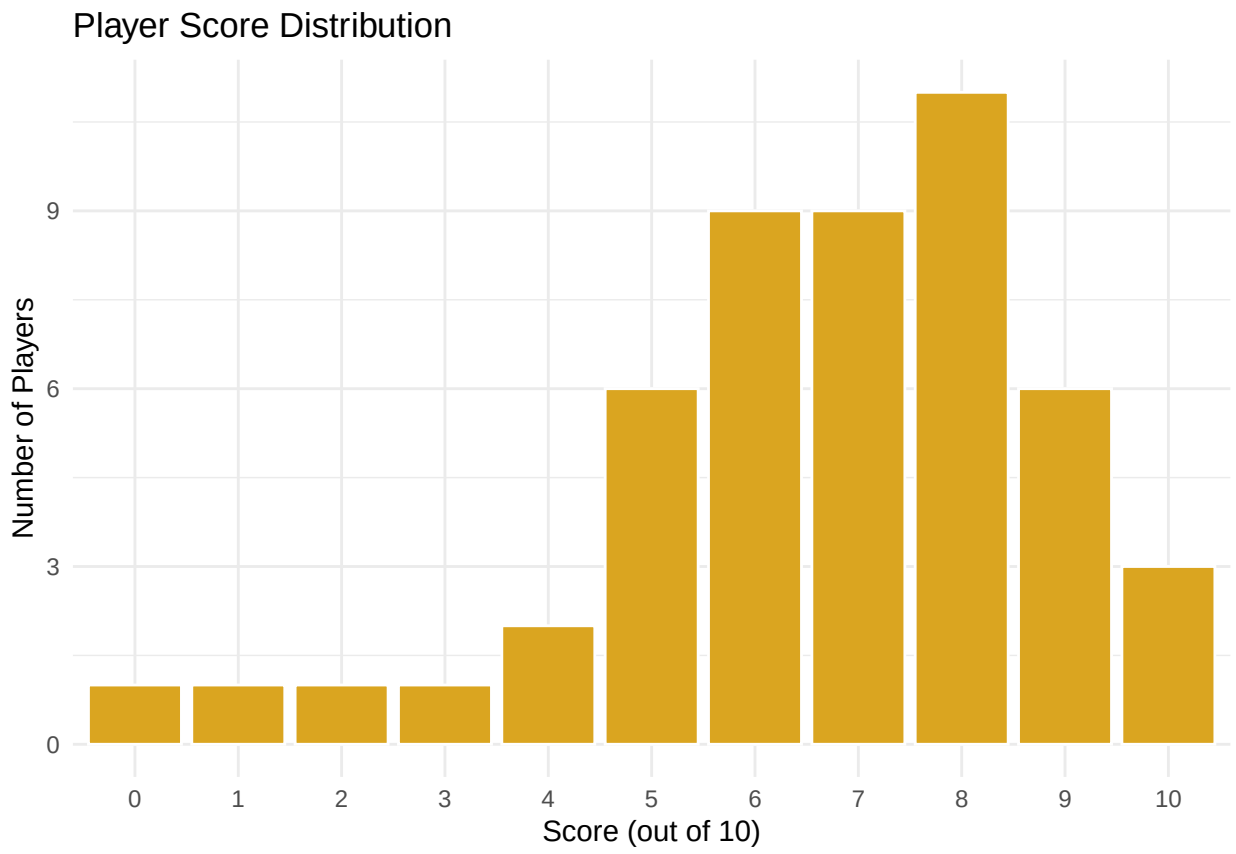
```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



7.Cybersecurity Awareness Game The Cybersecurity Awareness Game is an interactive module developed in-house that offers gamified learning for users. It presents cybersecurity quizzes or scenarios to players, allowing them to learn safe online practices through play and measuring their knowledge via scores.

a. Player Score Distribution This bar chart presents the distribution of player scores in the cybersecurity awareness game, with scores grouped from 0 to 10. It provides a quick snapshot of overall user performance and can help identify whether additional training or content refinement is required

```
set.seed(42)
scores_forced <- 0:10
scores_random <- rbinom(39, 10, 0.75)
game_scores <- sample(c(scores_forced, scores_random))
game_df <- tibble(Score = game_scores)

ggplot(game_df, aes(x = factor(Score, levels = 0:10))) +
  geom_bar(fill = "goldenrod", color = "white") +
  labs(title = "Player Score Distribution",
       x = "Score (out of 10)", y = "Number of Players") +
  theme_minimal()
```



PART 2

Included: CISA KEV entries pulled from HawkTalos /api/news/cisa Data source: CISA KEV catalog fetched through HawkTalos Data processing: exported to JSON then normalized into CSV (1464 rows) Analysis: frequency analysis (vendor/product), timelines (dateAdded), urgency (dueDate), CWE grouping

```
v <- read.csv("~/Desktop/vulns.csv", stringsAsFactors = FALSE, na.strings = c("", "NA"))

v$dateAdded <- as.Date(v$dateAdded)
v$dueDate   <- as.Date(v$dueDate)

total_vulns <- nrow(v)
total_vulns
```

```
## [1] 1464
```

1.Total vulnerabilities

This number is the total entries pulled from the CISA Known Exploited Vulnerabilities catalog through HawkTalos.

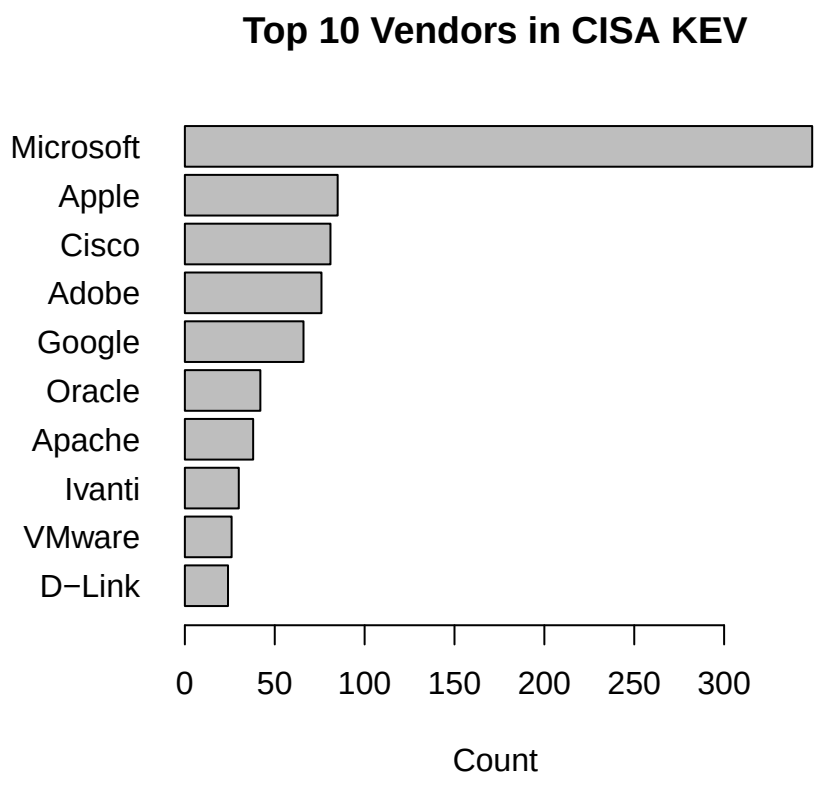
```
cat("Total vulnerabilities (CISA KEV entries):", total_vulns, "\n")
```

```
## Total vulnerabilities (CISA KEV entries): 1464
```

2.Top vendors

This chart shows which vendors appear most frequently in the KEV catalog. Higher counts can indicate widely used products or repeated exploitation of a vendor's software—useful for patch prioritization if your environment uses those vendors.

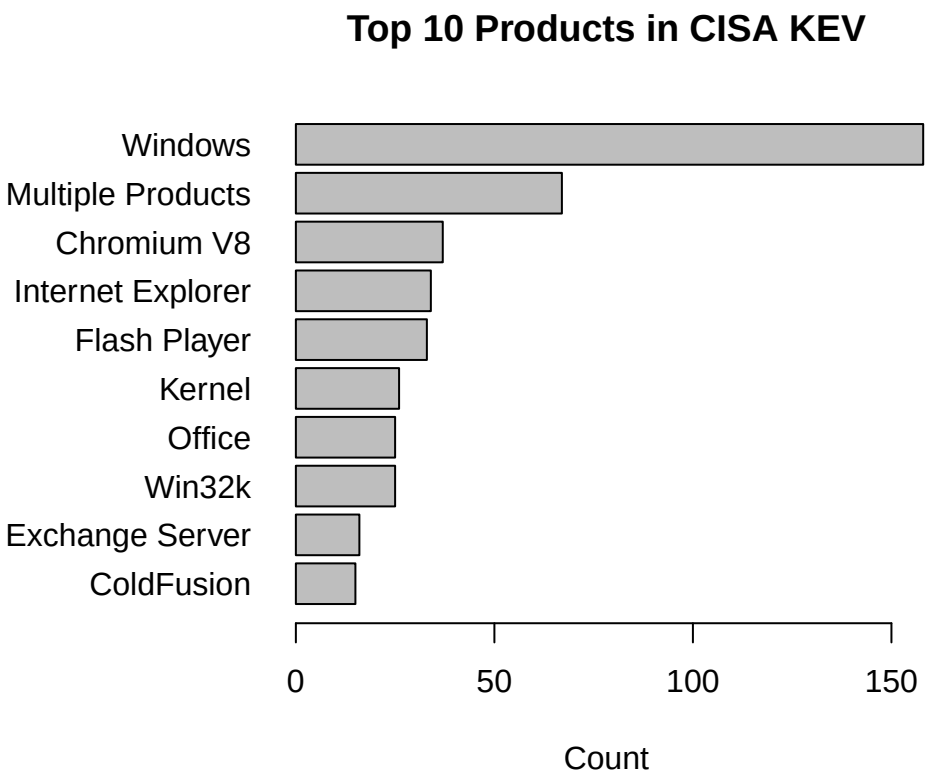
```
op <- par(no.readonly = TRUE); on.exit(par(op))
par(mar = c(5, 14, 4, 2))

top_vendors <- v$vendorProject |>
  table() |>
  sort(decreasing = TRUE) |>
  head(10)

barplot(
  rev(top_vendors),
  horiz = TRUE,
  las = 1,
  xlab = "Count",
  main = "Top 10 Vendors in CISA KEV"
)
```



3.Top products

This chart highlights which specific products show up most often in KEV. If any of these products exist in your organization, they become top candidates for immediate review and patching.

```r
op <- par(no.readonly = TRUE); on.exit(par(op))
par(mar = c(5, 14, 4, 2))

top_products <- v$product |>
  (\(x) x[!is.na(x) & x != ""])() |>
  table() |>
  sort(decreasing = TRUE) |>
  head(10)

barplot(
  rev(top_products),
  horiz = TRUE,
  las = 1,
  xlab = "Count",
  main = "Top 10 Products in CISA KEV"
)
```

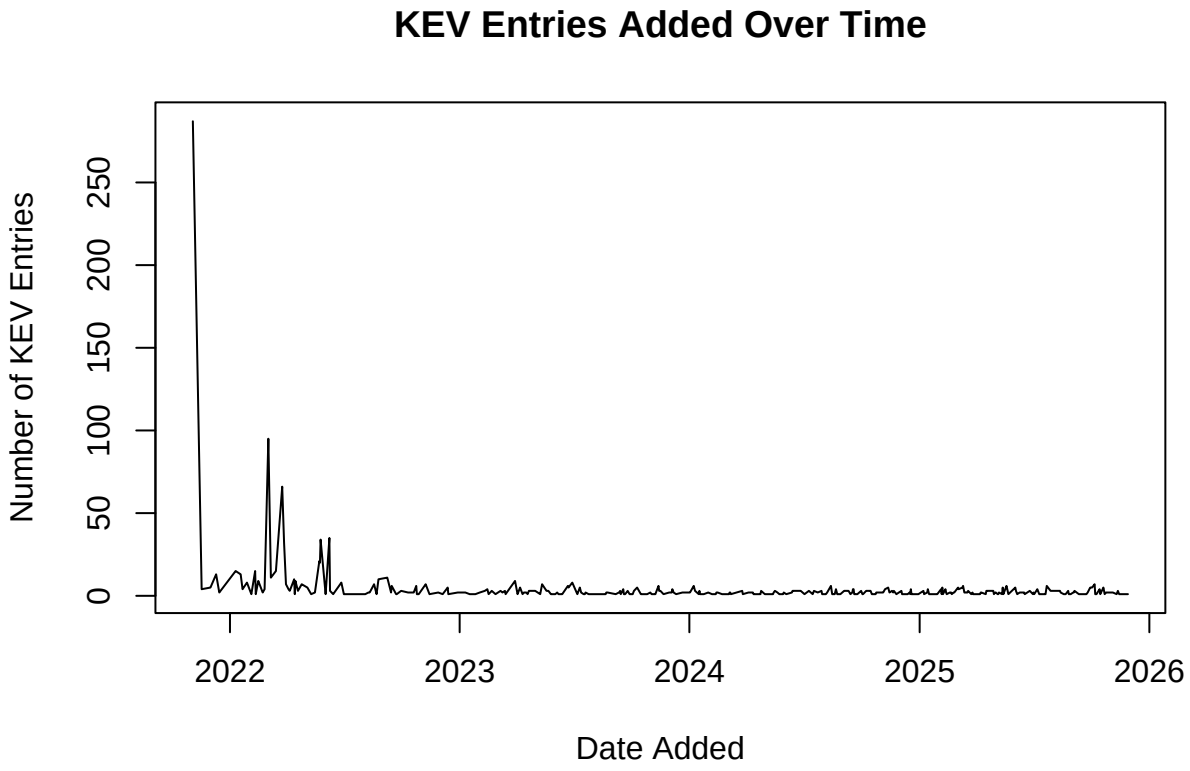


4.Vulnerabilities added over time

This timeline shows how frequently new KEV items were added over time.Spikes can reflect periods of increased exploitation activity or major vulnerability disclosures requiring urgent attention.

```r
added_counts <- v$dateAdded |>
  (\(d) d[!is.na(d)])() |>
  table()
```

```r
added_dates <- as.Date(names(added_counts))
added_n     <- as.integer(added_counts)
ord <- order(added_dates)

plot(
  added_dates[ord], added_n[ord],
  type = "l",
  xlab = "Date Added",
  ylab = "Number of KEV Entries",
  main = "KEV Entries Added Over Time"
)
```

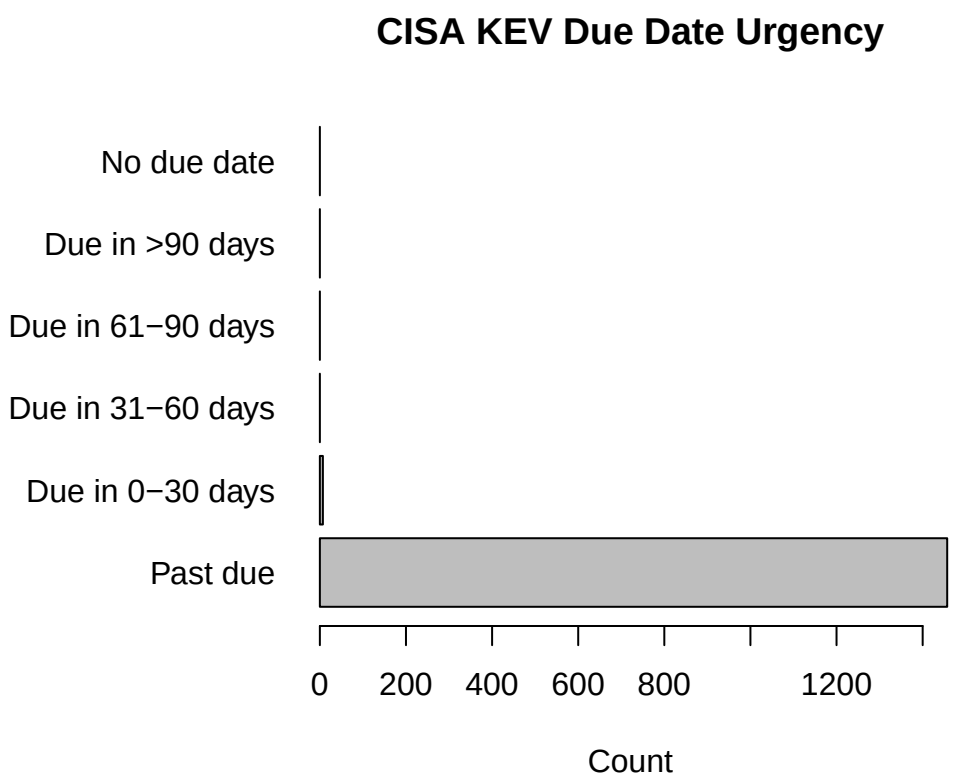


5.Due date urgency (30/60/90 days)

This chart groups KEV items by remediation urgency using CISA due dates. “Past due” and “Due in 0–30 days” represent the highest operational priority for patching or mitigation.

```r
today <- Sys.Date()

urgency <- ifelse(
  is.na(v$dueDate), "No due date",
  ifelse(v$dueDate < today, "Past due",
    ifelse(v$dueDate <= today + 30, "Due in 0-30 days",
      ifelse(v$dueDate <= today + 60, "Due in 31-60 days",
        ifelse(v$dueDate <= today + 90, "Due in 61-90 days",
               "Due in >90 days"
```

```
        )
      )
    )
  )
)

levels_order <- c(
  "Past due",
  "Due in 0-30 days",
  "Due in 31-60 days",
  "Due in 61-90 days",
  "Due in >90 days",
  "No due date"
)

urg_tab <- table(factor(urgency, levels = levels_order))

op <- par(no.readonly = TRUE); on.exit(par(op))
par(mar = c(5, 14, 4, 2))

barplot(
  urg_tab,
  horiz = TRUE,
  las = 1,
  xlab = "Count",
  main = "CISA KEV Due Date Urgency"
)
```

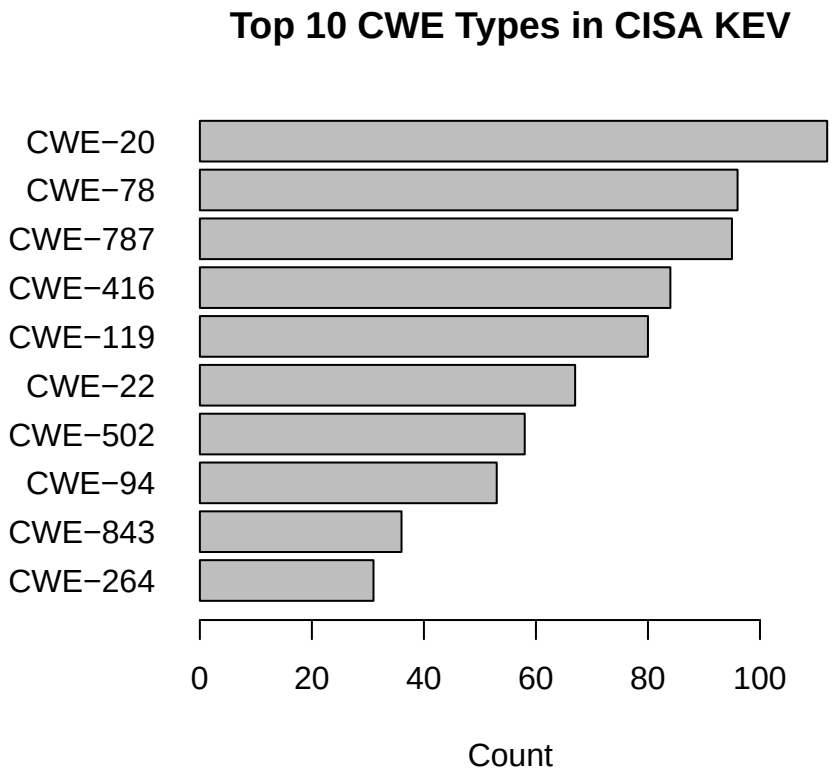


6.CWE distribution

This chart summarizes the most common weakness categories represented in the KEV catalog.It helps identify repeated vulnerability patterns that security training and secure coding practices should target.

```
cwe_vec <- v$cwes |>
  (\(x) x[!is.na(x) & x != ""])() |>
  strsplit(";") |>
  unlist(use.names = FALSE)

cwe_vec <- trimws(cwe_vec)
cwe_vec <- cwe_vec[cwe_vec != ""]

top_cwe <- cwe_vec |>
  table() |>
  sort(decreasing = TRUE) |>
  head(10)

op <- par(no.readonly = TRUE); on.exit(par(op))
par(mar = c(5, 14, 4, 2))

barplot(
  rev(top_cwe),
  horiz = TRUE,
  las = 1,
  xlab = "Count",
  main = "Top 10 CWE Types in CISA KEV"
```

```
)
```


Top 10 CWE Types in CISA KEV
CWE−20
CWE−78
CWE−787
CWE−416
CWE−119
CWE−22
CWE−502
CWE−94
CWE−843
CWE−264
0
20
40
60
80
100
Count